

Building a Multi-Page SPA with React Router

Easy Guide

Contents

1	Routing: Multiple Pages in Single-Page Applications	2
1.1	What is Routing?	2
1.2	Traditional Multi-Page Apps	2
1.3	Single-Page Applications (SPAs)	2
1.4	Why SPAs Are Better	3
1.5	How React Router Works	3
2	Installing React Router	4
2.1	Installation	4
2.2	Versions	4
2.2.1	Which to Choose?	4
2.3	Basic Setup	4
2.4	In main.jsx	5
3	Defining Routes	6
3.1	Basic Routes Setup	6
3.2	Understanding Routes	6
3.3	URL Structure	6
3.4	File Organization	7
3.5	Complete Example	7
4	Exploring an Alternative Way of Defining Routes	8
4.1	Configuration-Based Routing	8
4.2	Advantages	8
4.3	With Nested Routes	8
4.4	Comparing Approaches	9
5	Navigating between Pages with Links	10
5.1	Using Link Component	10
5.2	Why Link Instead of <a>?	10
5.3	Link vs Anchor	10
5.4	Navigation Example	11
6	Layouts & Nested Routes	12
6.1	The Problem	12
6.2	Nested Routes Solution	12
6.3	Outlet Component	12
6.4	Setting Up Nested Routes	13

6.5	How It Works	13
6.6	Result	13
7	Showing Error Pages with <code>errorElement</code>	14
7.1	Handling 404 Errors	14
7.2	Error Page Component	14
7.3	Catch-All Route	15
7.4	Complete Example	15
8	Working with Navigation Links (<code>NavLink</code>)	16
8.1	What is <code>NavLink</code> ?	16
8.2	Active Styling	16
8.3	Custom Styling	16
8.4	<code>NavLink</code> vs <code>Link</code>	17
8.5	Example: Navigation Menu	17
9	Defining & Using Dynamic Routes	18
9.1	Dynamic Route Basics	18
9.2	Syntax	18
9.3	Accessing Parameters	18
9.4	Multiple Parameters	19
9.5	Optional Parameters	19
10	Adding Links for Dynamic Routes	20
10.1	Linking to Dynamic Routes	20
10.2	Template Literals	20
10.3	From Dynamic Route	21
10.4	Links in Lists	21
11	Understanding Relative & Absolute Paths	22
11.1	Absolute Paths	22
11.2	Relative Paths	22
11.3	In React Router	22
11.4	When to Use Each	23
11.4.1	Use Absolute	23
11.4.2	Use Relative	23
11.5	Example	23
12	Working with Index Routes	24
12.1	What are Index Routes?	24
12.2	Defining Index Routes	24
12.3	When User Visits	24
12.4	In JSX Routes	25
12.5	Nested Index Routes	25

13 Data Fetching with a loader()	26
13.1 What is a loader()?	26
13.2 Creating a Loader	26
13.3 Using Loader in Route	26
13.4 Loader Parameters	27
14 Using Data From A Loader In The Route Component	28
14.1 Accessing Loader Data	28
14.2 Complete Example	28
14.3 No Loading Spinner Needed	29
15 Where Should loader() Code Be Stored?	30
15.1 File Organization	30
15.1.1 Option 1: Separate Loaders File	30
15.1.2 Option 2: With Components	31
15.1.3 Option 3: Inline in Router	31
15.2 Best Practice	31
16 When Are loader() Functions Executed?	32
16.1 Execution Timeline	32
16.2 When Exactly	32
16.3 Multiple Loaders	32
16.4 Parallel Loading	33
16.5 Don't Re-run on Every Render	33
17 Reflecting The Current Navigation State in the UI	34
17.1 Showing Loading State	34
17.2 Navigation States	34
17.3 Show Different UI	35
17.4 Get Current Location	35
17.5 Complete Example	35
18 Returning Responses in loader()	37
18.1 Returning Different Data Types	37
18.1.1 Return JSON	37
18.1.2 Return Raw Response	37
18.1.3 Return Object with Multiple Properties	37
18.2 Return With Metadata	38
18.3 Handling Errors	38
19 Which Kind Of Code Goes Into loader()	39
19.1 What Loaders Should Do	39
19.1.1 DO: Fetch Data	39
19.1.2 DO: Validate Access	39
19.1.3 DO: Check Authentication	39
19.2 What Loaders Should NOT Do	40
19.2.1 DON'T: Update State	40
19.2.2 DON'T: Run Side Effects	40
19.2.3 DON'T: Use React Hooks	40

19.3	Catching Errors	41
19.4	Error Component	41
19.5	Custom Errors	41
19.6	Different Errors for Different Routes	42
20	Extracting Error Data & Throwing Responses	43
20.1	Using <code>useRouteError</code>	43
20.2	Throwing Responses	43
20.3	Access Error Details	44
21	The <code>json()</code> Utility Function	45
21.1	What is <code>json()</code> ?	45
21.2	Why Use <code>json()</code> ?	45
21.3	With Status Codes	45
21.4	Complex Data	46
21.5	Comparison	46
22	Dynamic Routes & <code>loader()</code>	47
22.1	Combining Dynamic Routes and Loaders	47
22.2	In Component	47
22.3	Multiple Dynamic Parameters	48
22.4	Nested Dynamic Routes	48
23	The <code>useRouteLoaderData()</code> Hook & Accessing Data From Other Routes	49
23.1	Accessing Parent Loader Data	49
23.2	Complete Example	49
23.3	When to Use	50
24	Planning Data Submission	51
24.1	Form Submission Flow	51
24.2	Design Considerations	51
24.2.1	Question 1: Where to Process?	51
24.2.2	Question 2: Redirect After?	51
24.2.3	Question 3: Error Handling	52
24.3	Data Flow	52
25	Working with <code>action()</code> Functions	53
25.1	Creating an Action	53
25.2	Using in Route	53
25.3	In Component	54
25.4	HTTP Methods	54
26	Submitting Data Programmatically	55
26.1	Using <code>useNavigate</code>	55
26.2	Using <code>useFetcher</code>	55
26.3	Redirect After Submit	56
26.4	Stay on Same Page	56

27	Updating the UI State Based on the Submission Status	57
27.1	Tracking Submission State	57
27.2	Show Validation Status	57
27.3	Different States	58
27.4	With Action Data	58
28	Validating User Input & Outputting Validation Errors	59
28.1	Server-Side Validation	59
28.2	Display Errors	59
28.3	Highlight Fields	60
28.4	Common Validations	60
29	Reusing Actions via Request Methods	62
29.1	Handling Different Methods	62
29.2	Helper Functions	62
29.3	Route Setup	63
29.4	Form Setup	63
30	Behind-the-Scenes Work with useFetcher()	64
30.1	What is useFetcher?	64
30.2	Key Differences	64
30.3	Accessing Data	64
30.4	Real Example: Like Button	65
30.5	Multiple Fetchers	65
31	Deferring Data Fetching with defer()	67
31.1	What is defer()?	67
31.2	Using defer()	67
31.3	In Component	68
31.4	Benefits	68
32	defer() and json() with React Router v7 & Controlling Which Data Should Be Deferred	69
32.1	Using Both defer() and json()	69
32.2	Strategic Deferring	69
32.3	Decision Criteria	70
32.3.1	Load Immediately (json)	70
32.3.2	Defer Loading	70
32.4	Example Scenarios	70
32.5	React Router v7 Updates	71
32.6	Complete Example	71

Chapter 1

Routing: Multiple Pages in Single-Page Applications

1.1 What is Routing?

Routing is the system that handles navigating between different pages in your app without reloading the browser.

1.2 Traditional Multi-Page Apps

- User clicks a link
- Browser requests new HTML file from server
- Entire page reloads
- Slow and jarring experience
- Each page is separate file

1.3 Single-Page Applications (SPAs)

- User clicks a link
- JavaScript updates the page content
- No page reload
- URL changes but browser doesn't reload
- Feels like a smooth, fast app
- One HTML file for entire app

1.4 Why SPAs Are Better

1. **Speed** - No page reloads
2. **Smooth** - Transitions are instant
3. **Responsive** - Feels like native app
4. **Better UX** - No flashing screen
5. **Keep State** - Can maintain data between pages

1.5 How React Router Works

React Router intercepts link clicks and:

1. Changes the URL in address bar
2. Renders different component
3. Doesn't reload page
4. Updates browser history

Result: Users think they're on different pages, but it's all one app!

Chapter 2

Installing React Router

2.1 Installation

```
npm install react-router-dom
```

That's it! React Router is now available in your project.

2.2 Versions

- React Router v6 - Current stable version
- React Router v7 - Newest version with improvements

2.2.1 Which to Choose?

For new projects: Use the latest stable version.

2.3 Basic Setup

Import React Router in your main component:

```
import { BrowserRouter } from 'react-router-dom';

export default function App() {
  return (
    <BrowserRouter>
      {/* Your routes go here */}
    </BrowserRouter>
  );
}
```

BrowserRouter: The wrapper that enables routing in your app.

2.4 In main.jsx

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);
```

Important: Always wrap your app with BrowserRouter!

Chapter 3

Defining Routes

3.1 Basic Routes Setup

Use the `Routes` and `Route` components:

```
import { Routes, Route } from 'react-router-dom';
import Home from './pages/Home';
import About from './pages/About';
import Contact from './pages/Contact';

function App() {
  return (
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/about" element={<About />} />
      <Route path="/contact" element={<Contact />} />
    </Routes>
  );
}
```

3.2 Understanding Routes

Property	What It Does
path	URL path (e.g., “/about”)
element	Component to show

3.3 URL Structure

`http://example.com/about`
~~~~~  
path

When user visits `/about`, React Router renders the `About` component.

### 3.4 File Organization

```
/src
  /pages
    Home.jsx
    About.jsx
    Contact.jsx
  /components
  App.jsx
  main.jsx
```

### 3.5 Complete Example

```
import { Routes, Route } from 'react-router-dom';

function App() {
  return (
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/about" element={<About />} />
      <Route path="/services" element={<Services />} />
      <Route path="/contact" element={<Contact />} />
    </Routes>
  );
}

export default App;
```

**Path Matching:** Paths are matched in order, first match wins!

## Chapter 4

# Exploring an Alternative Way of Defining Routes

### 4.1 Configuration-Based Routing

Instead of JSX, use a configuration object:

```
import { createBrowserRouter, RouterProvider } from 'react-router-dom';

const routes = [
  { path: '/', element: <Home /> },
  { path: '/about', element: <About /> },
  { path: '/contact', element: <Contact /> }
];

const router = createBrowserRouter(routes);

function App() {
  return <RouterProvider router={router} />;
}
```

### 4.2 Advantages

- Cleaner separation from JSX
- Easier to manage many routes
- Better for complex routing
- Can be stored in separate files

### 4.3 With Nested Routes

```
const routes = [
  {
    path: '/',
```

```
    element: <Layout />,
    children: [
      { path: '/', element: <Home /> },
      { path: 'about', element: <About /> },
      { path: 'contact', element: <Contact /> }
    ]
  }
];
```

### 4.4 Comparing Approaches

| Feature     | JSX  | Config    |
|-------------|------|-----------|
| Simple      | Yes  | No        |
| Flexible    | Yes  | Yes       |
| Large apps  | No   | Yes       |
| Readability | Good | Very good |

**Choice:** Use JSX for simple apps, config for complex ones.

## Chapter 5

# Navigating between Pages with Links

### 5.1 Using Link Component

Navigate between routes using the Link component:

```
import { Link } from 'react-router-dom';

function Navigation() {
  return (
    <nav>
      <Link to="/">Home</Link>
      <Link to="/about">About</Link>
      <Link to="/contact">Contact</Link>
    </nav>
  );
}
```

### 5.2 Why Link Instead of <a>?

| Feature        | <a> | <Link> |
|----------------|-----|--------|
| Reloads page   | Yes | No     |
| Updates URL    | Yes | Yes    |
| Keeps state    | No  | Yes    |
| Smooth         | No  | Yes    |
| SPA experience | No  | Yes    |

### 5.3 Link vs Anchor

```
// BAD: Regular link reloads entire page
<a href="/about">About</a>

// GOOD: React Router link, no reload
<Link to="/about">About</Link>
```

## 5.4 Navigation Example

```
import { Link } from 'react-router-dom';

function App() {
  return (
    <div>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
        <Link to="/products">Products</Link>
        <Link to="/contact">Contact</Link>
      </nav>

      <main>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/about" element={<About />} />
          {/* ... more routes */}
        </Routes>
      </main>
    </div>
  );
}
```

**Rule:** Always use Link in SPAs, not anchor tags!



## Chapter 6

# Layouts & Nested Routes

### 6.1 The Problem

Every page needs navigation, header, footer. How to avoid repeating?

### 6.2 Nested Routes Solution

Create a layout component that wraps child pages:

```
function Layout() {
  return (
    <div>
      <Header />
      <nav>
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
      </nav>

      <main>
        <Outlet /> { /* Child pages render here */}
      </main>

      <Footer />
    </div>
  );
}
```

### 6.3 Outlet Component

Outlet is where child routes render.

```
<Layout>
  <Outlet /> { /* Child page renders here */}
</Layout>
```

## 6.4 Setting Up Nested Routes

```
import { Outlet } from 'react-router-dom';

const routes = [
  {
    path: '/',
    element: <Layout />,
    children: [
      { path: '/', element: <Home /> },
      { path: '/about', element: <About /> },
      { path: '/contact', element: <Contact /> }
    ]
  }
];
```

## 6.5 How It Works

User visits /about  
↓  
Matches /:  
↓  
Renders Layout component  
↓  
Layout has <Outlet />  
↓  
Outlet renders About component  
↓  
Result: Header + Nav + About + Footer

## 6.6 Result

Layout appears on every page. Only content changes:

|                   |                     |
|-------------------|---------------------|
| Header            | (same on all pages) |
| Nav   Nav   Nav   | (same on all pages) |
| Page Content Here | (changes per page)  |
| Footer            | (same on all pages) |

**Power Pattern:** Nested routes = shared layouts!

## Chapter 7

# Showing Error Pages with `errorElement`

### 7.1 Handling 404 Errors

When user visits non-existent page, show error page:

```
import ErrorPage from './pages/ErrorPage';

const routes = [
  {
    path: '/',
    element: <Layout />,
    errorElement: <ErrorPage />,
    children: [
      { path: '/', element: <Home /> },
      { path: '/about', element: <About /> }
    ]
  }
];
```

### 7.2 Error Page Component

```
import { useRouteError } from 'react-router-dom';

export default function ErrorPage() {
  const error = useRouteError();

  return (
    <div>
      <h1>Oops!</h1>
      <p>Sorry, an unexpected error occurred.</p>
      <p>{error.statusText || error.message}</p>
    </div>
  );
};
```

```
}
```

### 7.3 Catch-All Route

```
<Route path="*" element={<NotFound />} />
```

Matches any route that doesn't exist.

### 7.4 Complete Example

```
const routes = [  
  {  
    path: '/',  
    element: <Layout />,  
    errorElement: <ErrorPage />,  
    children: [  
      { path: '/', element: <Home /> },  
      { path: '/about', element: <About /> },  
      { path: '*', element: <NotFound /> } // Catch-all  
    ]  
  }  
];
```

**Important:** Always handle 404 errors gracefully!

## Chapter 8

# Working with Navigation Links (NavLink)

### 8.1 What is NavLink?

NavLink is like Link, but highlights when active.

```
import { NavLink } from 'react-router-dom';

function Navigation() {
  return (
    <nav>
      <NavLink to="/">Home</NavLink>
      <NavLink to="/about">About</NavLink>
      <NavLink to="/contact">Contact</NavLink>
    </nav>
  );
}
```

### 8.2 Active Styling

NavLink automatically adds active class when matching:

```
CSS:
a.active {
  background-color: blue;
  color: white;
}
```

### 8.3 Custom Styling

```
<NavLink
  to="/about"
  className={({ isActive }) => isActive ? 'active' : ''}
>
```

```
About
</NavLink>
```

Or use inline styles:

```
<NavLink
  to="/about"
  style={({ isActive }) => ({
    color: isActive ? 'blue' : 'black'
  })}
>
  About
</NavLink>
```

## 8.4 NavLink vs Link

| Feature           | Link | NavLink |
|-------------------|------|---------|
| Highlights active | No   | Yes     |
| Navigation        | Yes  | Yes     |
| Menu items        | No   | Yes     |
| Simple links      | Yes  | No      |

## 8.5 Example: Navigation Menu

```
<nav className="navbar">
  <NavLink to="/">Home</NavLink>
  <NavLink to="/products">Products</NavLink>
  <NavLink to="/about">About</NavLink>
  <NavLink to="/contact">Contact</NavLink>
</nav>
```

CSS:

```
.navbar a.active {
  border-bottom: 3px solid blue;
  font-weight: bold;
}
```

**Best Practice:** Use NavLink for navigation menus!

## Chapter 9

# Defining & Using Dynamic Routes

### 9.1 Dynamic Route Basics

Routes with variables in the path:

```
// Static route
/products

// Dynamic route
/products/123
/products/456
/user/john
/user/jane
```

### 9.2 Syntax

Use colons for variables:

```
<Route path="/products/:productId" element={<ProductDetail />} />
<Route path="/user/:userName" element={<UserProfile />} />
<Route path="/blog/:postId" element={<BlogPost />} />
```

The part after `:` is the parameter name.

### 9.3 Accessing Parameters

Use `useParams` hook:

```
import { useParams } from 'react-router-dom';

function ProductDetail() {
  const { productId } = useParams();

  return (
    <div>
      <h1>Product {productId}</h1>
    </div>
  );
}
```

```
    <p>Showing details for product {productId}</p>
  </div>
);
}
```

### 9.4 Multiple Parameters

```
<Route path="/user/:userId/post/:postId"
  element={<Post />} />

function Post() {
  const { userId, postId } = useParams();

  return (
    <div>
      <p>User: {userId}, Post: {postId}</p>
    </div>
  );
}
```

### 9.5 Optional Parameters

```
<Route path="/products/:productId?"
  element={<Products />} />

// Matches both:
/products          (productId = undefined)
/products/123      (productId = 123)
```

**Pattern:** Dynamic routes = flexible URLs!



## Chapter 10

# Adding Links for Dynamic Routes

### 10.1 Linking to Dynamic Routes

```
import { Link } from 'react-router-dom';

function ProductList() {
  const products = [
    { id: 1, name: 'Laptop' },
    { id: 2, name: 'Phone' },
    { id: 3, name: 'Tablet' }
  ];

  return (
    <ul>
      {products.map(product => (
        <li key={product.id}>
          <Link to={`/${product.id}`}>
            {product.name}
          </Link>
        </li>
      ))}
    </ul>
  );
}
```

### 10.2 Template Literals

Use backticks to insert variables:

```
// Static
to="/products"

// Dynamic
to={`/${productId}`}
```

```
// Multiple params
to={`~/user/${userId}/post/${postId}`}
```

## 10.3 From Dynamic Route

```
function ProductDetail() {
  const { productId } = useParams();

  const nextProductId = parseInt(productId) + 1;

  return (
    <div>
      <h1>Product {productId}</h1>
      <Link to={`~/products/${nextProductId}`}>
        Next Product
      </Link>
    </div>
  );
}
```

## 10.4 Links in Lists

```
function UserList() {
  const users = ['alice', 'bob', 'charlie'];

  return (
    <ul>
      {users.map(user => (
        <li key={user}>
          <Link to={`~/user/${user}`}>{user}</Link>
        </li>
      ))}
    </ul>
  );
}
```

**Tip:** Use template literals for dynamic URLs!

## Chapter 11

# Understanding Relative & Absolute Paths

### 11.1 Absolute Paths

Start from the root:

|           |              |
|-----------|--------------|
| /         | (home)       |
| /about    | (about page) |
| /user/123 | (user 123)   |

Full path from domain root.

### 11.2 Relative Paths

Relative to current location:

```
// From /admin/users:
.           = /admin/users
..          = /admin
../settings = /admin/users/settings
../dashboard = /admin/dashboard
```

### 11.3 In React Router

```
// Absolute path
<Link to="/products">Products</Link>

// Relative path
<Link to="../dashboard">Dashboard</Link>
<Link to="../settings">Settings</Link>
```

## 11.4 When to Use Each

### 11.4.1 Use Absolute

- Top-level navigation (menus)
- Links from anywhere
- User doesn't care about nesting

### 11.4.2 Use Relative

- Nested page navigation
- Moving up directory structure
- Context-sensitive links

## 11.5 Example

```
// Navigation menu (absolute)
<Link to="/">Home</Link>
<Link to="/products">Products</Link>

// In product detail page (relative)
<Link to=".">Back to Products</Link>
<Link to="../../categories">All Categories</Link>
```

**Default:** Absolute paths are simpler and more predictable!

## Chapter 12

# Working with Index Routes

### 12.1 What are Index Routes?

Routes that render at parent path without additional segment:

```
// Index route
/           (renders Home)
/about      (renders About)

// Without index route
/about
/about/team    (would need index route)
/about/history
```

### 12.2 Defining Index Routes

Omit the path or use `index: true`:

```
const routes = [
  {
    path: '/',
    element: <Layout />,
    children: [
      { index: true, element: <Home /> },
      { path: 'about', element: <About /> },
      { path: 'contact', element: <Contact /> }
    ]
  }
];
```

### 12.3 When User Visits

```
/           → Renders Layout + Home
/about      → Renders Layout + About
/contact    → Renders Layout + Contact
```

## 12.4 In JSX Routes

```
<Routes>
  <Route path="/" element={<Layout />}>
    <Route index element={<Home />} />
    <Route path="about" element={<About />} />
    <Route path="contact" element={<Contact />} />
  </Route>
</Routes>
```

## 12.5 Nested Index Routes

```
{
  path: '/dashboard',
  element: <DashboardLayout />,
  children: [
    { index: true, element: <Overview /> },
    { path: 'analytics', element: <Analytics /> },
    { path: 'reports', element: <Reports /> }
  ]
}

// /dashboard renders Overview
// /dashboard/analytics renders Analytics
```

**Pattern:** Index routes = default content for a path!

## Chapter 13

# Data Fetching with a loader()

### 13.1 What is a loader()?

A function that fetches data before a route renders.

Old way:

1. Route renders
2. `useEffect` runs
3. Fetches data
4. Shows loading spinner
5. Then shows data

New way (with loader):

1. Fetch data first
2. Then render route
3. Data already loaded
4. No loading spinner needed

### 13.2 Creating a Loader

```
export async function productLoader({ params }) {
  const response = await fetch(
    `/api/products/${params.productId}`
  );
  return response.json();
}
```

### 13.3 Using Loader in Route

```
import { productLoader } from './loaders';

const routes = [
  {
    path: '/products/:productId',
    element: <ProductDetail />,
  },
];
```

```
    loader: productLoader // Pass loader here
  }
];
```

### 13.4 Loader Parameters

```
async function productLoader({ params, request }) {
  const { productId } = params; // Dynamic params

  return {
    id: productId,
    data: await fetchProduct(productId)
  };
}
```

Available in loader:

- `params` - Dynamic route parameters
- `request` - The request object
- `context` - Custom context

**Advantage:** Data loads before component renders!



## Chapter 14

# Using Data From A Loader In The Route Component

### 14.1 Accessing Loader Data

Use `useLoaderData` hook:

```
import { useLoaderData } from 'react-router-dom';

export default function ProductDetail() {
  const product = useLoaderData();

  return (
    <div>
      <h1>{product.name}</h1>
      <p>Price: ${product.price}</p>
      <p>{product.description}</p>
    </div>
  );
}
```

### 14.2 Complete Example

Loader:

```
export async function productLoader({ params }) {
  const response = await fetch(
    `/api/products/${params.productId}`
  );
  if (!response.ok) throw new Error('Product not found');
  return response.json();
}
```

Component:

```
export default function ProductDetail() {
  const product = useLoaderData();

  return (
    <div className="product">
      <img src={product.image} alt={product.name} />
      <h1>{product.name}</h1>
      <p className="price">${product.price}</p>
      <p className="description">{product.description}</p>
      <button>Add to Cart</button>
    </div>
  );
}
```

Route:

```
{
  path: '/products/:productId',
  element: <ProductDetail />,
  loader: productLoader
}
```

## 14.3 No Loading Spinner Needed

Since data loads before render:

```
// Don't need this!
const [loading, setLoading] = useState(true);
const [data, setData] = useState(null);

useEffect(() => {
  fetchData().then(data => {
    setData(data);
    setLoading(false);
  });
}, []);

if (loading) return <Spinner />;
```

Data is already there!

**Clean Code:** No more useState and useEffect for loading data!

## Chapter 15

# Where Should loader() Code Be Stored?

### 15.1 File Organization

#### 15.1.1 Option 1: Separate Loaders File

```
/src
  /loaders
    productLoaders.js
    userLoaders.js
  /pages
    ProductDetail.jsx
  router.js
```

**loaders/productLoaders.js:**

```
export async function productLoader({ params }) {
  const response = await fetch(
    `/api/products/${params.productId}`
  );
  return response.json();
}
```

**router.js:**

```
import { productLoader } from './loaders/productLoaders';

const routes = [
  {
    path: '/products/:productId',
    element: <ProductDetail />,
    loader: productLoader
  }
];
```

### 15.1.2 Option 2: With Components

```
/src
  /pages
    ProductDetail.jsx
    ProductDetail.loaders.js
```

**ProductDetail.jsx:**

```
export default function ProductDetail() {
  const product = useLoaderData();
  return <div>{product.name}</div>;
}
```

// In router file:

```
import { productLoader } from './pages/ProductDetail.loaders';
```

### 15.1.3 Option 3: Inline in Router

```
const routes = [
  {
    path: '/products/:productId',
    element: <ProductDetail />,
    loader: async ({ params }) => {
      const response = await fetch(
        `/api/products/${params.productId}`
      );
      return response.json();
    }
  }
];
```

## 15.2 Best Practice

1. Small projects: Inline or with component
2. Large projects: Separate loaders file
3. Many loaders: Organize by feature
4. Share logic: Extract to utilities

**Recommendation:** Separate loaders file for clean organization!

## Chapter 16

# When Are loader() Functions Executed?

### 16.1 Execution Timeline

```
User clicks link to /products/123
↓
React Router sees path change
↓
Finds matching route with loader
↓
Calls loader({ params, request })
↓
Waits for loader to finish
↓
Data loaded, route renders
↓
Component receives data via useLoaderData()
```

### 16.2 When Exactly

Loaders run:

- When route matches
- **Before** component renders
- After user clicks link
- Before displaying page

### 16.3 Multiple Loaders

If nested routes have loaders:

```
{
```

```
path: '/dashboard',
loader: dashboardLoader, // Runs first
element: <Dashboard />,
children: [
  {
    path: 'analytics',
    loader: analyticsLoader, // Runs too
    element: <Analytics />
  }
]
}
```

// Both loaders run, then components render

### 16.4 Parallel Loading

React Router can load multiple loaders at once:

```
// If both match, they load in parallel
Time: 1s (both loaders at same time)
```

Instead of:

```
Time: 1s + 1s = 2s (one after another)
```

### 16.5 Don't Re-run on Every Render

```
// Loaders only run when route changes
// Not on component re-render
// Not on state changes in component

// This is different from useEffect:
useEffect(() => {
  // Runs on component render, state changes
}, [someState]);
```

**Timing:** Loaders run once per route change, before render!

## Chapter 17

# Reflecting The Current Navigation State in the UI

### 17.1 Showing Loading State

Use `useNavigation` hook:

```
import { useNavigation } from 'react-router-dom';

function App() {
  const navigation = useNavigation();

  return (
    <div>
      {navigation.state === 'loading' && (
        <div className="loading-bar">
          Loading...
        </div>
      )}

      <main>
        <Outlet />
      </main>
    </div>
  );
}
```

### 17.2 Navigation States

| State      | What It Means        |
|------------|----------------------|
| idle       | Not navigating       |
| loading    | Loaders running      |
| submitting | Form being submitted |

## 17.3 Show Different UI

```
const navigation = useNavigation();

if (navigation.state === 'loading') {
  return <Spinner />;
}

if (navigation.state === 'submitting') {
  return <p>Saving changes...</p>;
}

return <NormalUI />;
```

## 17.4 Get Current Location

```
const navigation = useNavigation();

// While loading
console.log(navigation.location.pathname);
// /products/123

// See where user is going
const isGoingToProducts =
  navigation.location?.pathname?.includes('/products');
```

## 17.5 Complete Example

```
function Layout() {
  const navigation = useNavigation();
  const isLoading = navigation.state === 'loading';

  return (
    <div>
      <Header />

      {isLoading && (
        <div className="loading-bar">
          <p>Loading page...</p>
        </div>
      )}

      <nav>
        <NavLink to="/">Home</NavLink>
        <NavLink to="/products">Products</NavLink>
      </nav>
    </div>
  );
}
```



```
    <main>
      <Outlet />
    </main>

    <Footer />
  </div>
);
}
```

**UX:** Always show users when loading is happening!

## Chapter 18

# Returning Responses in loader()

### 18.1 Returning Different Data Types

#### 18.1.1 Return JSON

```
export async function productLoader({ params }) {
  const response = await fetch(
    `/api/products/${params.productId}`
  );
  return response.json(); // Parse JSON
}
```

#### 18.1.2 Return Raw Response

```
export async function productLoader({ params }) {
  const response = await fetch(
    `/api/products/${params.productId}`
  );
  return response; // Return response object
}
```

```
// Component parses it
const product = await useLoaderData().json();
```

#### 18.1.3 Return Object with Multiple Properties

```
export async function dashboardLoader() {
  return {
    user: await fetchUser(),
    stats: await fetchStats(),
    notifications: await fetchNotifications()
  };
}
```

```
// Component
const { user, stats, notifications } = useLoaderData();
```

## 18.2 Return With Metadata

```
export async function productLoader({ params }) {
  const product = await fetch(
    `/api/products/${params.productId}`
  ).then(r => r.json());

  return {
    product,
    timestamp: new Date(),
    cached: false
  };
}
```

## 18.3 Handling Errors

```
export async function productLoader({ params }) {
  try {
    const response = await fetch(
      `/api/products/${params.productId}`
    );

    if (!response.ok) {
      throw new Response('Not Found', { status: 404 });
    }

    return response.json();
  } catch (error) {
    throw new Response(error.message, { status: 500 });
  }
}
```

**Flexibility:** Return whatever the component needs!

## Chapter 19

# Which Kind Of Code Goes Into loader()

### 19.1 What Loaders Should Do

#### 19.1.1 DO: Fetch Data

```
export async function userLoader({ params }) {  
  const response = await fetch(`/api/user/${params.userId}`);  
  return response.json();  
}
```

#### 19.1.2 DO: Validate Access

```
export async function adminLoader() {  
  const user = await getUser();  
  if (user.role !== 'admin') {  
    throw new Response('Unauthorized', { status: 403 });  
  }  
  return user;  
}
```

#### 19.1.3 DO: Check Authentication

```
export async function protectedLoader() {  
  const isLoggedIn = await checkAuth();  
  if (!isLoggedIn) {  
    return redirect('/login');  
  }  
  return null;  
}
```

## 19.2 What Loaders Should NOT Do

### 19.2.1 DON'T: Update State

```
// Bad - can't use useState
export async function badLoader() {
  const [user, setUser] = useState(null); // ERROR!
  return user;
}
```

### 19.2.2 DON'T: Run Side Effects

```
// Bad - avoid side effects
export async function badLoader() {
  localStorage.setItem('visited', true); // Avoid
  return data;
}
```

### 19.2.3 DON'T: Use React Hooks

```
// Bad - this is not a component
export async function badLoader() {
  const data = useContext(DataContext); // ERROR!
  return data;
}
```

\end{troubleshooting}

\section{Best Practices}

```
\begin{enumerate}
  \item Fetch data
  \item Validate user permissions
  \item Transform data if needed
  \item Throw errors if problems
  \item Redirect if needed
  \item Return prepared data
\end{enumerate}
```

\textcolor[HTML]{E74C3C}{\textbf{Rule:}} Loaders are server-side-like code, not React code!

% ===== Chapter 20 =====

\chapter{Error Handling with Custom Errors}

\section{Throwing Errors in Loaders}

```
\begin{verbatim}
export async function productLoader({ params }) {
  const response = await fetch(
    `/api/products/${params.productId}`
  )
}
```

```
);

if (!response.ok) {
  throw new Response('Product not found', {
    status: 404
  });
}

return response.json();
}
```

## 19.3 Catching Errors

Use `errorElement` to show error page:

```
const routes = [
  {
    path: '/products/:productId',
    element: <ProductDetail />,
    loader: productLoader,
    errorElement: <ErrorPage />
  }
];
```

## 19.4 Error Component

```
import { useRouteError } from 'react-router-dom';

export default function ErrorPage() {
  const error = useRouteError();

  return (
    <div>
      <h1>Error!</h1>
      <p>{error.status} - {error.statusText}</p>
      <p>{error.message}</p>

      <button onClick={() => window.location.href = '/'}>
        Go Home
      </button>
    </div>
  );
}
```

## 19.5 Custom Errors

```
class ValidationError extends Error {
```

```
    constructor(message, status) {
      super(message);
      this.status = status;
    }
  }

  export async function userLoader({ params }) {
    if (!params.userId) {
      throw new ValidationError('User ID required', 400);
    }

    // ... rest of loader
  }
```

## 19.6 Different Errors for Different Routes

```
const routes = [
  {
    path: '/',
    element: <Layout />,
    errorElement: <GeneralError />,
    children: [
      {
        path: 'admin',
        loader: adminLoader,
        errorElement: <AdminError />
      },
      {
        path: 'products/:id',
        loader: productLoader,
        errorElement: <ProductError />
      }
    ]
  }
];
```

**Strategy:** Specific error pages for specific routes!

## Chapter 20

# Extracting Error Data & Throwing Responses

### 20.1 Using useRouteError

Get error information:

```
import { useRouteError } from 'react-router-dom';

export default function ErrorBoundary() {
  const error = useRouteError();

  return (
    <div>
      <h1>{error.status}</h1>
      <p>{error.statusText}</p>
      <p>{error.message}</p>
    </div>
  );
}
```

### 20.2 Throwing Responses

```
export async function productLoader({ params }) {
  if (!params.productId) {
    throw new Response(
      JSON.stringify({ message: 'ID required' }),
      { status: 400, statusText: 'Bad Request' }
    );
  }

  const response = await fetch(
    `/api/products/${params.productId}`
  );
}
```



```
if (!response.ok) {
  throw response; // Throw response directly
}

return response.json();
}
```

### 20.3 Access Error Details

```
import { useRouteError } from 'react-router-dom';

export default function ErrorPage() {
  const error = useRouteError();

  let message = 'An unexpected error occurred';
  let status = 500;

  if (error.status === 404) {
    message = 'Page not found';
  } else if (error.status === 403) {
    message = 'You do not have permission';
  }

  return (
    <div className={`error-${error.status}`}>
      <h1>Error {error.status}</h1>
      <p>{message}</p>
    </div>
  );
}
```

**Pattern:** Throw responses with proper status codes!

## Chapter 21

# The json() Utility Function

### 21.1 What is json()?

Helper function to return JSON responses:

```
import { json } from 'react-router-dom';

export async function productLoader({ params }) {
  const product = await fetch(
    `/api/products/${params.productId}`
  ).then(r => r.json());

  return json(product);
}
```

### 21.2 Why Use json()?

```
// Instead of:
return response.json();
```

```
// Use:
return json(response);
```

Benefits:

- Sets correct content-type header
- Consistent with errors
- Better for SSR
- Explicit intent

### 21.3 With Status Codes

```
import { json } from 'react-router-dom';
```

```
export async function userLoader({ params }) {
  const user = await fetchUser(params.userId);

  if (!user) {
    throw json(
      { message: 'User not found' },
      { status: 404 }
    );
  }

  return json(user, { status: 200 });
}
```

### 21.4 Complex Data

```
return json({
  user: userData,
  posts: userPosts,
  followers: followerCount,
  verified: true
});
```

### 21.5 Comparison

```
// Old way
export async function loader() {
  const data = await fetch('/api/data').then(r => r.json());
  return data;
}

// New way
import { json } from 'react-router-dom';

export async function loader() {
  const data = await fetch('/api/data').then(r => r.json());
  return json(data);
}
```

**Best Practice:** Use `json()` for clarity and consistency!

## Chapter 22

# Dynamic Routes & loader()

### 22.1 Combining Dynamic Routes and Loaders

```
export async function productLoader({ params }) {
  const { productId } = params;

  const response = await fetch(
    `/api/products/${productId}`
  );

  if (!response.ok) {
    throw new Response('Product not found', { status: 404 });
  }

  return response.json();
}

// Route definition
{
  path: '/products/:productId',
  element: <ProductDetail />,
  loader: productLoader,
  errorElement: <ErrorPage />
}
```

### 22.2 In Component

```
import { useLoaderData, useParams } from 'react-router-dom';

export default function ProductDetail() {
  const product = useLoaderData();
  const { productId } = useParams();

  return (
    <div>
```

```
    <h1>{product.name}</h1>
    <p>ID: {productId}</p>
    <p>Price: ${product.price}</p>
  </div>
);
}
```

## 22.3 Multiple Dynamic Parameters

```
export async function postLoader({ params }) {
  const { userId, postId } = params;

  const response = await fetch(
    `/api/users/${userId}/posts/${postId}`
  );

  return response.json();
}

// Route
{
  path: '/users/:userId/posts/:postId',
  loader: postLoader,
  element: <PostDetail />
}
```

## 22.4 Nested Dynamic Routes

```
{
  path: '/users/:userId',
  loader: userLoader,
  element: <UserLayout />,
  children: [
    {
      index: true,
      element: <UserProfile />
    },
    {
      path: 'posts/:postId',
      loader: postLoader,
      element: <PostDetail />
    }
  ]
}
```

**Power:** Dynamic routes + loaders = clean data fetching!

## Chapter 23

# The useRouteLoaderData() Hook & Accessing Data From Other Routes

### 23.1 Accessing Parent Loader Data

Sometimes child needs parent's data:

```
import { useRouteLoaderData } from 'react-router-dom';

export default function Child() {
  // Get parent route's loader data
  const parentData = useRouteLoaderData('parent-route');

  return <div>{parentData.name}</div>;
}
\end{relevantinfo}

\section{ID-based Access}
```

First, give route an ID:

```
\begin{verbatim}
{
  id: 'user-route',
  path: '/users/:userId',
  loader: userLoader,
  element: <UserPage />
}
```

Then access from anywhere:

```
const userData = useRouteLoaderData('user-route');
```

### 23.2 Complete Example

```
// Router definition
```

```
const routes = [
  {
    id: 'user',
    path: '/users/:userId',
    loader: userLoader,
    element: <UserLayout />,
    children: [
      {
        path: 'posts',
        element: <Posts />
      }
    ]
  }
];

// In Posts component
function Posts() {
  const user = useRouteLoaderData('user');

  return (
    <div>
      <p>Showing posts for {user.name}</p>
    </div>
  );
}
```

### 23.3 When to Use

- Child needs parent data
- Avoid passing through props
- Deeply nested components
- Shared data across routes

**Alternative:** Avoid prop drilling with this hook!

## Chapter 24

# Planning Data Submission

### 24.1 Form Submission Flow

```
User fills form
↓
User clicks submit
↓
Form data collected
↓
Send to action() function
↓
Action processes data
↓
Update database
↓
Return result
↓
UI updates or redirect
```

### 24.2 Design Considerations

#### 24.2.1 Question 1: Where to Process?

- Client (browser) - Fast but not secure
- Server (action) - Secure but slower
- Both - Validation on both

#### 24.2.2 Question 2: Redirect After?

- After success - Better UX
- Stay on page - Show confirmation
- Go to specific page - Redirect elsewhere



### 24.2.3 Question 3: Error Handling

- Show errors on same page
- Highlight invalid fields
- Clear error messages

## 24.3 Data Flow

```
Form
↓
<Form action="/action">
↓
action() function
↓
Validate data
↓
Save to database
↓
Return success or errors
↓
Component receives result
↓
Update UI or redirect
```

**Planning:** Think through the entire submission flow!

## Chapter 25

# Working with action() Functions

### 25.1 Creating an Action

An action function handles form submissions:

```
export async function createProductAction({ request }) {
  const formData = await request.formData();

  const product = {
    name: formData.get('name'),
    price: formData.get('price'),
    description: formData.get('description')
  };

  // Validate
  if (!product.name) {
    return { error: 'Name is required' };
  }

  // Save to database
  const result = await fetch('/api/products', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(product)
  });

  if (!result.ok) {
    return { error: 'Failed to create product' };
  }

  return result.json();
}
```

### 25.2 Using in Route

```
const routes = [
```

```
{
  path: '/products/new',
  element: <NewProduct />,
  action: createProductAction
}
];
```

## 25.3 In Component

```
import { Form, useActionData } from 'react-router-dom';

export default function NewProduct() {
  const actionData = useActionData();

  return (
    <div>
      {actionData?.error && (
        <p className="error">{actionData.error}</p>
      )}

      <Form method="post">
        <input name="name" type="text" />
        <input name="price" type="number" />
        <textarea name="description"></textarea>
        <button type="submit">Create</button>
      </Form>
    </div>
  );
}
```

## 25.4 HTTP Methods

```
// Create (POST)
<Form method="post" action="/products/new">

// Update (PUT)
<Form method="put" action={`\products/${id}`}>

// Delete (DELETE)
<Form method="delete" action={`\products/${id}`}>
```

**Pattern:** Form method matches action function!

## Chapter 26

# Submitting Data Programmatically

### 26.1 Using useNavigate

Navigate and submit programmatically:

```
import { useNavigate } from 'react-router-dom';

function SendData() {
  const navigate = useNavigate();

  const handleSubmit = async () => {
    const response = await fetch('/api/data', {
      method: 'POST',
      body: JSON.stringify({ data: 'value' })
    });

    if (response.ok) {
      navigate('/success');
    }
  };

  return <button onClick={handleSubmit}>Submit</button>;
}
```

### 26.2 Using useFetcher

Submit without navigation:

```
import { useFetcher } from 'react-router-dom';

function LikeButton() {
  const fetcher = useFetcher();

  return (
    <fetcher.Form method="post" action="/like">
      <input type="hidden" name="postId" value={postId} />
    </fetcher.Form>
  );
}
```

```
        <button type="submit">Like</button>
      </fetcher.Form>
    );
  }
}
```

## 26.3 Redirect After Submit

```
import { redirect } from 'react-router-dom';

export async function submitAction({ request }) {
  const formData = await request.formData();

  // Process form
  const result = await saveData(formData);

  if (result.ok) {
    return redirect('/success');
  }

  return { error: 'Failed' };
}
```

## 26.4 Stay on Same Page

```
export async function submitAction({ request }) {
  const formData = await request.formData();

  // Process
  const result = await saveData(formData);

  // Return message, don't redirect
  return {
    message: 'Saved successfully',
    data: result
  };
}
```

**Choice:** Redirect for page changes, stay for confirmations!

## Chapter 27

# Updating the UI State Based on the Submission Status

### 27.1 Tracking Submission State

Use `useNavigation`:

```
import { useNavigation } from 'react-router-dom';

function MyForm() {
  const navigation = useNavigation();

  const isSubmitting =
    navigation.state === 'submitting';

  return (
    <Form method="post">
      <input name="title" />
      <button disabled={isSubmitting}>
        {isSubmitting ? 'Saving...' : 'Save'}
      </button>
    </Form>
  );
}
```

### 27.2 Show Validation Status

```
const navigation = useNavigation();
const formData = navigation.formData;

if (formData) {
  const title = formData.get('title');
  console.log('Saving:', title);
}
```

## 27.3 Different States

```
const navigation = useNavigation();

if (navigation.state === 'idle') {
  return <NormalForm />;
}

if (navigation.state === 'submitting') {
  return <SavingForm />;
}

if (navigation.state === 'loading') {
  return <LoadingForm />;
}
```

## 27.4 With Action Data

```
function Form() {
  const actionData = useActionData();
  const navigation = useNavigation();

  return (
    <form method="post">
      {actionData?.error && (
        <p className="error">{actionData.error}</p>
      )}

      {actionData?.success && (
        <p className="success">Saved!</p>
      )}

      <button disabled={navigation.state === 'submitting'}>
        Submit
      </button>
    </form>
  );
}
```

**UX:** Always give feedback during submission!

## Chapter 28

# Validating User Input & Outputting Validation Errors

### 28.1 Server-Side Validation

```
export async function createUserAction({ request }) {
  const formData = await request.formData();

  const errors = {};
  const email = formData.get('email');
  const password = formData.get('password');

  // Validation
  if (!email) errors.email = 'Email required';
  if (!email.includes('@')) errors.email = 'Invalid email';
  if (!password) errors.password = 'Password required';
  if (password.length < 8) errors.password = 'Too short';

  // Return errors
  if (Object.keys(errors).length > 0) {
    return { errors };
  }

  // Save user
  const user = await createUser(email, password);
  return user;
}
```

### 28.2 Display Errors

```
import { Form, useActionData } from 'react-router-dom';

function RegisterForm() {
  const actionData = useActionData();
```



```
return (
  <Form method="post">
    <div>
      <input name="email" type="email" />
      {actionData?.errors?.email && (
        <p className="error">{actionData.errors.email}</p>
      )}
    </div>

    <div>
      <input name="password" type="password" />
      {actionData?.errors?.password && (
        <p className="error">{actionData.errors.password}</p>
      )}
    </div>

    <button type="submit">Register</button>
  </Form>
);
}
```

## 28.3 Highlight Fields

```
function RegisterForm() {
  const actionData = useActionData();

  return (
    <Form method="post">
      <input
        name="email"
        className={actionData?.errors?.email ? 'error' : ''}
      />
      <span className="error">
        {actionData?.errors?.email}
      </span>
    </Form>
  );
}
```

```
CSS:
input.error {
  border: 2px solid red;
}
```

## 28.4 Common Validations

```
// Email
```

```
if (!email.includes('@')) {
  errors.email = 'Invalid email format';
}

// Password strength
if (password.length < 8) {
  errors.password = 'Must be at least 8 characters';
}

// Matching fields
if (password !== confirmPassword) {
  errors.password = 'Passwords do not match';
}

// Unique value
const existing = await User.findOne({ email });
if (existing) {
  errors.email = 'Email already in use';
}
```

**Critical:** Always validate on the server!

## Chapter 29

# Reusing Actions via Request Methods

### 29.1 Handling Different Methods

One action for POST, PUT, DELETE:

```
export async function productAction({ request, params }) {
  const { productId } = params;

  if (request.method === 'POST') {
    return createProduct(request);
  }

  if (request.method === 'PUT') {
    return updateProduct(productId, request);
  }

  if (request.method === 'DELETE') {
    return deleteProduct(productId);
  }
}
```

### 29.2 Helper Functions

```
async function createProduct(request) {
  const formData = await request.formData();
  const product = {
    name: formData.get('name'),
    price: formData.get('price')
  };

  return await fetch('/api/products', {
    method: 'POST',
    body: JSON.stringify(product)
  });
}
```

```
async function updateProduct(id, request) {
  const formData = await request.formData();

  return await fetch(`/api/products/${id}`, {
    method: 'PUT',
    body: JSON.stringify(Object.fromEntries(formData))
  });
}

async function deleteProduct(id) {
  return await fetch(`/api/products/${id}`, {
    method: 'DELETE'
  });
}
```

## 29.3 Route Setup

```
const routes = [
  {
    path: '/products',
    action: productAction
  },
  {
    path: '/products/:productId',
    action: productAction
  }
];
```

## 29.4 Form Setup

```
// Create
<Form method="post" action="/products">

// Update
<Form method="put" action={` /products/${id}`}>

// Delete
<Form method="delete" action={` /products/${id}`}>
```

**Pattern:** One action handles all operations!

## Chapter 30

# Behind-the-Scenes Work with useFetcher()

### 30.1 What is useFetcher?

Submit data without navigation:

```
import { useFetcher } from 'react-router-dom';

function LikeButton({ postId }) {
  const fetcher = useFetcher();

  return (
    <fetcher.Form method="post" action="/like">
      <input type="hidden" name="postId" value={postId} />
      <button type="submit">Like</button>
    </fetcher.Form>
  );
}
```

### 30.2 Key Differences

|              | Form | useFetcher |
|--------------|------|------------|
| Navigation   | Yes  | No         |
| URL change   | Yes  | No         |
| Redirect     | Yes  | No         |
| Async action | Yes  | Yes        |
| Data update  | Yes  | Yes        |

### 30.3 Accessing Data

```
const fetcher = useFetcher();

// Form element
fetcher.Form
```

```
// Current state
fetcher.state // 'idle', 'submitting', 'loading'

// Data from action
fetcher.data

// Form data being submitted
fetcher.formData

// HTTP method
fetcher.formMethod // 'POST', 'PUT', etc.
```

## 30.4 Real Example: Like Button

```
function Post({ post }) {
  const fetcher = useFetcher();

  const isLiking = fetcher.state === 'submitting';
  const hasLiked = fetcher.data?.liked;

  return (
    <div className="post">
      <p>{post.content}</p>

      <fetcher.Form method="post" action="/like">
        <input type="hidden" name="postId" value={post.id} />
        <button disabled={isLiking}>
          {hasLiked ? ' Liked' : ' Like'}
        </button>
      </fetcher.Form>
    </div>
  );
}
```

## 30.5 Multiple Fetchers

```
function Post({ post }) {
  const likeFetcher = useFetcher();
  const commentFetcher = useFetcher();

  return (
    <div>
      <likeFetcher.Form method="post">
        { /* Like form */ }
      </likeFetcher.Form>
    </div>
  );
}
```

```
    <commentFetcher.Form method="post">
      {/* Comment form */}
    </commentFetcher.Form>
  </div>
);
}
```

**Use Case:** Like buttons, comments, small forms!

## Chapter 31

# Deferring Data Fetching with defer()

### 31.1 What is defer()?

Render page while data loads in background:

```
// Old way:  
User clicks link  
  ↓  
Loader fetches data  
  ↓  
Wait for data  
  ↓  
Then render page
```

```
// New way (with defer):  
User clicks link  
  ↓  
Render page immediately  
  ↓  
Data loads in background  
  ↓  
Show data when ready
```

### 31.2 Using defer()

```
import { defer } from 'react-router-dom';  
  
export async function dashboardLoader() {  
  // Load immediately  
  const user = await fetchUser();  
  
  // Load later  
  const analytics = fetchAnalytics();  
  const reports = fetchReports();  
}
```



```
return defer({
  user,
  analytics,
  reports
});
}
```

### 31.3 In Component

```
import { Await, useLoaderData } from 'react-router-dom';
import { Suspense } from 'react';

export default function Dashboard() {
  const { user, analytics, reports } = useLoaderData();

  return (
    <div>
      <h1>Hello {user.name}</h1>

      <Suspense fallback={<p>Loading analytics...</p>}>
        <Await resolve={analytics}>
          {(data) => <Analytics data={data} />}
        </Await>
      </Suspense>

      <Suspense fallback={<p>Loading reports...</p>}>
        <Await resolve={reports}>
          {(data) => <Reports data={data} />}
        </Await>
      </Suspense>
    </div>
  );
}
```

### 31.4 Benefits

- Faster initial page render
- Show content as it loads
- Better perceived performance
- Progressive enhancement

**Pattern:** `defer()` for fast UX!

## Chapter 32

# defer() and json() with React Router v7 & Controlling Which Data Should Be Deferred

### 32.1 Using Both defer() and json()

```
import { defer, json } from 'react-router-dom';

export async function productLoader({ params }) {
  // Load immediately (json)
  const product = json(await fetchProduct(params.id));

  // Load later (defer)
  const reviews = fetchReviews(params.id);
  const recommendations = fetchRecommendations(params.id);

  return defer({
    product,
    reviews,
    recommendations
  });
}
```

### 32.2 Strategic Deferring

Choose what to defer and what not to:

```
// Fast - load immediately
async function getUser() {
  return await fetchUser(); // Use json()
}

// Slow - defer loading
async function getAnalytics() {
```

```
    return await fetchAnalytics(); // Use defer()
  }

export async function dashboardLoader() {
  return defer({
    user: json(await getUser()),
    analytics: getAnalytics()
  });
}
```

## 32.3 Decision Criteria

### 32.3.1 Load Immediately (json)

- Data is fast to fetch
- Critical for page display
- Small data size
- User expects it

### 32.3.2 Defer Loading

- Data is slow to fetch
- Not critical for initial view
- Can show placeholder
- Non-blocking data

## 32.4 Example Scenarios

```
// E-commerce product page
defer({
  product: json(quickData),      // Load first
  reviews: fetchReviews(),      // Defer
  recommendations: fetchRecs(), // Defer
  inventory: json(quickData)    // Load first
});
```

```
// Social media profile
defer({
  profile: json(quickData),      // Load first
  posts: fetchPosts(),          // Defer
  followers: fetchFollowers(),  // Defer
  following: fetchFollowing()    // Defer
});
```

## 32.5 React Router v7 Updates

React Router v7 improved `defer()`:

- Better error handling
- Streaming support
- Performance improvements
- Cleaner API

## 32.6 Complete Example

```
import { defer, json, useLoaderData, Await } from 'react-router-dom';
import { Suspense } from 'react';

export async function productLoader({ params }) {
  const product = await fetchProduct(params.id);

  return defer({
    product: json(product),
    reviews: fetchReviews(params.id),
    related: fetchRelatedProducts(params.id)
  });
}

export default function ProductPage() {
  const { product, reviews, related } = useLoaderData();

  return (
    <div>
      <h1>{product.name}</h1>

      <Suspense fallback=<p>Loading reviews...</p>>
        <Await resolve={reviews}>
          {(data) => <Reviews items={data} />}
        </Await>
      </Suspense>

      <Suspense fallback=<p>Loading recommendations...</p>>
        <Await resolve={related}>
          {(data) => <RelatedProducts items={data} />}
        </Await>
      </Suspense>
    </div>
  );
}
```

**Strategy:** Load critical data first, defer the rest!

# Quick Summary

## React Router Essentials

1. **Routes** - Define pages
2. **Links** - Navigate between pages
3. **Dynamic Routes** - Variable URLs
4. **Loaders** - Fetch data before render
5. **Actions** - Handle form submissions
6. **Fetchers** - Submit without navigation

## Complete Flow

```
User clicks link
↓
Router finds matching route
↓
Loader runs (fetch data)
↓
Component renders with data
↓
User submits form
↓
Action function processes
↓
Result returned to component
↓
UI updates or redirect
```

## Best Practices

1. Always use Link, not <a>
2. Use NavLink for navigation menus
3. Organize routes in separate file

4. Keep loaders and actions separate
5. Validate on server
6. Show loading states
7. Handle errors gracefully
8. Defer slow-loading data

## File Organization

```
/src
  /pages
    Home.jsx
    About.jsx
  /loaders
    productLoaders.js
  /actions
    formActions.js
  /components
  router.js
  App.jsx
```

## Key Hooks

| Hook            | Purpose                   |
|-----------------|---------------------------|
| useNavigate()   | Programmatic navigation   |
| useParams()     | Get dynamic parameters    |
| useLoaderData() | Access loader data        |
| useActionData() | Access action results     |
| useNavigation() | Track navigation state    |
| useFetcher()    | Submit without navigation |

## Common Patterns

1. Nested routes + Outlet = Shared layout
2. Dynamic routes + params = Detail pages
3. Loaders + defer = Progressive rendering
4. Actions + validation = Form handling
5. Fetcher = Like buttons, comments

*React Router makes building SPAs intuitive and powerful!*